

Curso Profesional de JasperReports 6.20.0 Community

Módulo 4 — Parámetros y lógica

TEORÍA

AUTOR: JAIME GALLO

JasperReports Library 6.20.0 Community · Jaspersoft Studio 6.20.0 Community Edition · Java 8 · Maven · SQLite · CSV · XML · JSON · Proyecto EditorialReports

Módulo 4 — Parámetros y lógica

Curso: Curso Profesional de JasperReports 6.20.0 Community

Proyecto acumulativo: EditorialReports

Baseline: M3/3.7 validado end-to-end

Este documento conserva los seis puntos originales del material recibido y corrige únicamente las divergencias técnicas detectadas contra JasperReports 6.20.0 y contra el baseline ejecutable. Las correcciones principales afectan a parámetros, LEFT JOIN, tratamiento de null, fuentes portables y parametrización SQL.

Punto 4.1 — Parámetros

Módulo, proyecto y objetivos de aprendizaje

Módulo: 4 — Parámetros y lógica (3 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial

Punto: 4.1 — Parámetros

Objetivos de aprendizaje

- Declarar parámetros con tipo Java y valor por defecto, y distinguirlos de la inicialización propia de las variables.
- Distinguir los parámetros de usuario de los parámetros internos del motor.
- Utilizar los parámetros internos (REPORT_PARAMETERS_MAP, REPORT_CONNECTION, REPORT_LOCALE, REPORT_TIME_ZONE).
- Configurar la propiedad isForPrompting y el diálogo de solicitud de parámetros en Jaspersoft Studio.
- Combinar parámetros en expresiones de texto, de fecha y de cálculo.
- Pasar parámetros desde código Java y desde el diálogo de previsualización.
- Documentar los parámetros del proyecto EditorialReports.

Parte teórica

Bloque 1 — Declaración de parámetros con tipo y valor por defecto

Un parámetro de informe se declara con `<parameter>`, un name único y una clase Java. Su valor llega normalmente desde el `Map<String, Object>` utilizado durante el llenado. Si el llamador no proporciona una entrada para ese nombre, JasperReports usa `defaultValueExpression` cuando existe; en caso contrario el valor es `null`. En JasperReports 6.20.0 los parámetros no tienen `initialValueExpression`: esa expresión pertenece al ciclo de vida de las variables.

XML

```
<parameter name="fechaDesde" class="java.util.Date">
  <defaultValueExpression><![CDATA[new java.util.Date(0)]]></defaultValueExpression>
</parameter>
<parameter name="fechaHasta" class="java.util.Date">
  <defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression>
</parameter>
<parameter name="rangoFechas" class="java.lang.String">
  <defaultValueExpression><![CDATA[
    new java.text.SimpleDateFormat("dd/MM/yyyy").format(${fechaDesde})
    + " - " +
    new java.text.SimpleDateFormat("dd/MM/yyyy").format(${fechaHasta})
  ]]></defaultValueExpression>
</parameter>
```

Línea 1-3	fechaDesde declara un parámetro Date con fecha inicial por defecto.
Línea 4-6	fechaHasta utiliza la fecha actual cuando Java no proporciona otra.
Línea 7-13	rangoFechas demuestra que un defaultValueExpression puede usar parámetros previamente declarados.

La evaluación de un valor por defecto no sustituye un valor que sí haya sido entregado en el mapa. Por eso el diseño puede ofrecer defaults útiles para Preview mientras el programa Java conserva la capacidad de sobrescribirlos explícitamente.

Bloque 2 — Parámetros de usuario y parámetros internos

JasperReports distingue dos tipos de parámetros. Los parámetros de usuario son los que declara el diseñador en el JRXML y que el programa Java proporciona. Los parámetros internos son los que el propio motor inyecta en el mapa de parámetros y que están disponibles sin declaración. Los parámetros internos más utilizados son REPORT_PARAMETERS_MAP, REPORT_CONNECTION, REPORT_LOCALE, REPORT_TIME_ZONE, REPORT_RESOURCE_BUNDLE, REPORT_DATA_SOURCE, REPORT_SCRIPTLET y REPORT_MAX_COUNT. El diseñador puede referenciar los parámetros internos con la sintaxis `${}` sin necesidad de declararlos.

```
<textFieldExpression><![CDATA["Zona horaria: " + ${REPORT_TIME_ZONE}.getID()]]></textFieldExpression>
```

Línea 1

```
<textFieldExpression><![CDATA["Zona horaria: " + ${REPORT_TIME_ZONE}.getID()]]></textFieldExpression>
```

→ la expresión referencia el parámetro interno `REPORT_TIME_ZONE` sin declararlo. El parámetro es una instancia de `java.util.TimeZone` y su método `getID()` devuelve el identificador.

El parámetro `REPORT_PARAMETERS_MAP` contiene el mapa completo de parámetros que el programa Java ha proporcionado al motor. Su utilidad es permitir que una expresión acceda a los parámetros por nombre en tiempo de ejecución, incluso si no están declarados en el JRXML. El parámetro `REPORT_CONNECTION` contiene la conexión JDBC que el motor ha recibido. El parámetro `REPORT_LOCALE` contiene la configuración regional del informe, que se puede utilizar en las expresiones para formatear fechas y números según el idioma. El parámetro `REPORT_TIME_ZONE` contiene la zona horaria. El parámetro `REPORT_RESOURCE_BUNDLE` contiene el paquete de recursos para la internacionalización. Los parámetros internos están disponibles en todas las bandas del informe.

PARÁMETROS INTERNOS DE JASPERREPORTS 6.20.0

<code>REPORT_PARAMETERS_MAP</code>	→ Mapa de parámetros proporcionado por el programa
<code>REPORT_CONNECTION</code>	→ Conexión JDBC utilizada por el motor
<code>REPORT_DATA_SOURCE</code>	→ Fuente de datos utilizada por el motor
<code>REPORT_LOCALE</code>	→ Configuración regional del informe
<code>REPORT_TIME_ZONE</code>	→ Zona horaria del informe
<code>REPORT_RESOURCE_BUNDLE</code>	→ Paquete de recursos para la internacionalización
<code>REPORT_SCRIPTLET</code>	→ Scriptlet asociado al informe
<code>REPORT_MAX_COUNT</code>	→ Número máximo de registros a procesar
<code>REPORT_FILE_RESOLVER</code>	→ Resolutor de rutas de archivo
<code>REPORT_CLASS_LOADER</code>	→ Cargador de clases utilizado por el motor
<code>REPORT_VIRTUALIZER</code>	→ Virtualizador del documento

Qué representa el diagrama: los parámetros internos más utilizados de JasperReports 6.20.0. Están disponibles sin necesidad de declaración.

Por qué es relevante: permite aprovechar la información que el motor proporciona sin tener que pasarla explícitamente desde el programa Java.

Bloque 3 — La propiedad `isForPrompting`

La propiedad `isForPrompting` controla si un parámetro se solicita al usuario en el diálogo de previsualización de Jaspersoft Studio. Su valor por defecto es `true`, lo que significa que el parámetro aparece en el diálogo. Si se establece a `false`, el parámetro no aparece en el diálogo y se utiliza su valor por defecto o el valor proporcionado por el programa Java. Esta propiedad resulta útil para los parámetros que no deben ser modificados por el usuario, como los parámetros internos o los parámetros calculados a partir de otros. La propiedad se declara en el elemento `parameter` junto al nombre y la clase.

```
<parameter name="REPORT_TIME_ZONE" class="java.util.TimeZone" isForPrompting="false"/>
<parameter name="usuario" class="java.lang.String" isForPrompting="true"/>
```

Línea 1

```
<parameter name="REPORT_TIME_ZONE" class="java.util.TimeZone" isForPrompting="false"/>
```

→ declara el parámetro interno con `isForPrompting="false"`. El parámetro no aparece en el diálogo de previsualización. En la práctica, este parámetro no se declara porque el motor lo proporciona automáticamente; el ejemplo ilustra el uso de la propiedad.

Línea 2

```
<parameter name="usuario" class="java.lang.String" isForPrompting="true"/>
```

→ declara el parámetro usuario con `isForPrompting="true"`. El parámetro aparece en el diálogo de previsualización y el usuario puede proporcionar un valor.

El diálogo de previsualización de Jaspersoft Studio muestra una pestaña con los parámetros que tienen `isForPrompting="true"`. El usuario puede rellenar los valores y pulsar OK para ejecutar el informe con esos valores. Si un parámetro tiene valor por defecto, el diálogo lo muestra precargado y el usuario puede modificarlo o aceptarlo. Si un parámetro no tiene valor por defecto y tiene `isForPrompting="true"`, el diálogo muestra un campo vacío y el usuario debe rellenarlo. La propiedad `isForPrompting` describe si el parámetro está destinado a ser solicitado por herramientas de diseño; el llenado programático sigue recibiendo sus valores mediante el mapa de parámetros.

COMPORTAMIENTO DE `isForPrompting`

```
isForPrompting="true" (por defecto):
```

- El parámetro aparece en el diálogo de previsualización.
- El usuario puede proporcionar un valor.
- Si tiene valor por defecto, aparece precargado.

```
isForPrompting="false":
```

- El parámetro NO aparece en el diálogo de previsualización.
- El usuario no puede proporcionar un valor desde el entorno.
- El valor se toma del `defaultValueExpression` o del programa Java.

Qué representa el diagrama: el comportamiento de la propiedad `isForPrompting`. La propiedad solo afecta al diálogo de Jaspersoft Studio, no al motor.

Por qué es relevante: permite controlar qué parámetros se solicitan al usuario en el entorno de diseño y cuáles se resuelven automáticamente.

Bloque 4 — Parámetros en expresiones de texto, fecha y cálculo

Los parámetros pueden utilizarse en expresiones de tres tipos. Las expresiones de texto concatenan el valor del parámetro con literales o con otros parámetros. Las expresiones de fecha formatean el valor del parámetro con un patrón. Las expresiones de cálculo realizan operaciones aritméticas con el valor del parámetro y con otros valores. La combinación de los tres tipos permite construir informes que se adaptan a las instrucciones del usuario y que presentan los datos con el formato adecuado.

```
<textFieldExpression><![CDATA["Informe de " + $P{departamento} + " - " + $P{periodo}]]></textFieldExpression>
```

Línea 1	<code><textFieldExpression><![CDATA[. . .]]></textFieldExpression></code> → expresión de texto que concatena dos parámetros con literales.
Línea 1 (continuación)	<code>"Informe de " + \$P{departamento}</code> → concatena el literal con el parámetro <code>departamento</code> .
Línea 1 (continuación)	<code>+ " - " + \$P{periodo}</code> → añade el literal y el parámetro <code>periodo</code> .

Los parámetros también pueden utilizarse en expresiones de cálculo. Una expresión que calcula el importe total con IVA puede multiplicar un parámetro `tipoIva` por el valor de un campo. Una expresión que calcula el descuento puede multiplicar un parámetro `porcentajeDescuento` por el importe total. Una expresión que calcula la fecha de vencimiento puede sumar un parámetro `diasVencimiento` a la fecha actual. La combinación de parámetros con campos permite construir cálculos que dependen tanto de los datos como de las instrucciones del usuario.

```
<textFieldExpression><![CDATA[$F{importe_total} * (1 + $P{tipoIva})]]></textFieldExpression>
```

Línea 1	<code><textFieldExpression><![CDATA[\$F{importe_total} * (1 + \$P{tipoIva})]]></textFieldExpression></code> → expresión que multiplica el importe total por el factor <code>(1 + tipoIva)</code> . Si <code>tipoIva</code> es 0.21, el resultado es el importe con IVA incluido.
----------------	--

TRES TIPOS DE EXPRESIONES CON PARÁMETROS

```
Texto:
  "Informe de " + $P{departamento}
  "Generado por " + $P{usuario} + " el " + $P{fecha}

Fecha:
  new SimpleDateFormat("dd/MM/yyyy").format($P{fechaDesde})
  $P{fechaDesde}.before($P{fechaHasta})

Cálculo:
  $F{importe_total} * (1 + $P{tipoIva})
  $F{precio} - ($F{precio} * $P{descuento})
  $V{TotalImporte} / $P{numElementos}
```

Qué representa el diagrama: los tres tipos de expresiones con parámetros. Cada tipo resuelve un caso de uso distinto.

Por qué es relevante: permite identificar el patrón adecuado para cada necesidad. La combinación de los tres tipos cubre la mayoría de los casos.

Bloque 5 — Parámetros en consultas SQL y paso desde Java

Los parámetros pueden utilizarse en las consultas SQL declaradas en el elemento `queryString`. La sintaxis es `$P{nombreDelParametro}` y el motor sustituye la expresión por el valor del parámetro antes de enviar la consulta a la base de datos. El tipo del parámetro determina la forma de la sustitución. Los parámetros de tipo `String` se sustituyen con comillas simples. Los parámetros numéricos se sustituyen sin comillas. Los parámetros de fecha se sustituyen con el formato que el motor considere adecuado. Los parámetros en las consultas SQL se estudian en detalle en el punto 4.6; en este punto se introduce su sintaxis para que las consultas del informe sean completas.

```
<queryString language="sql">
  <![CDATA[
    SELECT titulo, precio, paginas
```

```
FROM libros
WHERE precio >= $P{precioMinimo}
AND precio <= $P{precioMaximo}
ORDER BY titulo
]]>
</queryString>
```

Línea 4	WHERE precio >= \$P{precioMinimo} → filtro por precio mínimo. JasperReports enlaza \$P{precioMinimo} como parámetro de un PreparedStatement; no concatena el valor en el texto SQL.
Línea 5	AND precio <= \$P{precioMaximo} → filtro por precio máximo.
Línea 6	ORDER BY titulo → ordenación por título.

El paso de parámetros desde Java se realiza mediante el mapa de parámetros que se proporciona al método `fillReport`. El mapa es un `Map<String, Object>` donde las claves son los nombres de los parámetros y los valores son los objetos que se pasan. La coherencia entre el nombre del parámetro en el mapa y el nombre declarado en el JRXML es condición necesaria para que el valor se asigne correctamente. El mapa puede contener más entradas que parámetros declarados; las entradas adicionales se ignoran. El mapa puede contener menos entradas que parámetros declarados; los parámetros ausentes se resuelven con su valor por defecto o con un error si no lo tienen.

```
Map<String, Object> parametros = new HashMap<>();
parametros.put("usuario", "Ana Martínez");
parametros.put("fechaInforme", new java.util.Date());
parametros.put("precioMinimo", 15.0);
parametros.put("precioMaximo", 25.0);

JasperPrint documento = JasperFillManager.fillReport(
    rutaJasper, parametros, conexion);
```

Línea 1	Map<String, Object> parametros = new HashMap<>(); → declara el mapa de parámetros.
Línea 2	parametros.put("usuario", "Ana Martínez"); → introduce el valor del parámetro usuario.
Línea 3	parametros.put("fechaInforme", new java.util.Date()); → introduce el valor del parámetro fechaInforme.
Línea 4	parametros.put("precioMinimo", 15.0); → introduce el valor del parámetro precioMinimo.
Línea 5	parametros.put("precioMaximo", 25.0); → introduce el valor del parámetro precioMaximo.
Línea 7-8	JasperFillManager.fillReport(rutaJasper, parametros, conexion); → llena el informe con el mapa de parámetros y la conexión.

Resumen rápido de la teoría

- Los parámetros se declaran con `parameter` y los atributos `name` y `class`.
- El `defaultValueExpression` define el valor cuando el programa no proporciona ninguno.
- Los parámetros no disponen de `initialValueExpression`; esa expresión pertenece a las variables. En un parámetro se utiliza `defaultValueExpression` cuando se necesita un valor por defecto.
- Los parámetros internos como `REPORT_PARAMETERS_MAP` y `REPORT_CONNECTION` están disponibles sin declaración.
- La propiedad `isForPrompting` controla la aparición del parámetro en el diálogo de previsualización.
- Los parámetros se utilizan en expresiones de texto, fecha y cálculo.
- Los parámetros se utilizan en consultas SQL con la sintaxis `$P{}`.
- El paso de parámetros desde Java se realiza con un `Map<String, Object>`.

Punto 4.2 — Filtros con parámetros

Módulo, proyecto y objetivos de aprendizaje

Módulo: 4 — Parámetros y lógica (3 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial

Punto: 4.2 — Filtros con parámetros

Objetivos de aprendizaje

- Aplicar filtros estáticos y dinámicos en las consultas SQL mediante parámetros.
- Utilizar filtros opcionales con la técnica del parámetro nulo.
- Filtrar filas en la banda Detail mediante `printWhenExpression`.
- Construir filtros combinados con operadores lógicos.
- Diferenciar el filtrado en SQL del filtrado en la plantilla.
- Documentar los filtros del proyecto EditorialReports.

Parte teórica

Bloque 1 — Filtrado en SQL frente a filtrado en la plantilla

Un informe con parámetros puede filtrar los datos en dos momentos distintos del ciclo. El primero es la consulta SQL: la cláusula `WHERE` restringe las filas que el motor recibe del `ResultSet`. El segundo es la plantilla: la propiedad `printWhenExpression` decide si un registro ya recibido se imprime o se omite. El filtrado en SQL reduce el volumen de datos que viaja desde la base de datos hasta el motor y es más eficiente. El filtrado en la plantilla permite aplicar condiciones que dependen del contexto del informe y que no pueden expresarse en SQL. La elección entre ambos depende del tipo de filtro y del volumen de datos.

```
<queryString language="sql">
  <![CDATA[
    SELECT titulo, precio, paginas
    FROM libros
    WHERE precio >= $P{precioMinimo}
  ]]>
</queryString>
```

Línea 3	<code>SELECT titulo, precio, paginas</code> → indica las columnas que se recuperan.
Línea 4	<code>FROM libros</code> → indica la tabla de origen.
Línea 5	<code>WHERE precio >= \$P{precioMinimo}</code> → filtra las filas. Solo se recuperan los libros cuyo precio sea mayor o igual que el valor del parámetro.

El filtrado en SQL tiene tres ventajas. La primera es el rendimiento: la base de datos aplica el filtro utilizando sus índices y devuelve solo las filas necesarias. La segunda es la reducción de memoria: el motor recibe menos registros y construye un documento en memoria más pequeño. La tercera es la simplicidad del informe: la plantilla no necesita contener lógica de filtrado. La contrapartida es que el filtro debe expresarse en SQL, lo que limita las condiciones a las que el motor de base de datos puede evaluar. Los filtros que dependen de la configuración regional, del idioma del usuario o del estado del informe no pueden expresarse en SQL.

FILTRADO EN SQL vs FILTRADO EN LA PLANTILLA

Filtrado en SQL:

- Se aplica en la cláusula `WHERE` de la consulta.
- Utiliza los índices de la base de datos.
- Reduce el volumen de datos que viaja al motor.
- No puede acceder al estado del informe.

Filtrado en la plantilla:

- Se aplica con `printWhenExpression` en cada banda.
- No utiliza índices.
- Recibe todos los datos y luego los descarta.
- Puede acceder al estado del informe.

Qué representa el diagrama: las diferencias entre el filtrado en SQL y el filtrado en la plantilla. Cada uno tiene ventajas y limitaciones distintas.

Por qué es relevante: permite elegir el momento adecuado para cada filtro. Los filtros simples y de alto volumen se aplican en SQL. Los filtros complejos o dependientes del contexto se aplican en la plantilla.

Bloque 2 — Filtros opcionales con parámetro nulo

Un filtro opcional es un filtro que se aplica solo cuando el usuario proporciona un valor para el parámetro. La técnica más habitual consiste en utilizar un parámetro que puede ser nulo y una cláusula WHERE que comprueba si el parámetro tiene valor. Cuando el parámetro es nulo, el filtro no se aplica. Cuando el parámetro tiene un valor, el filtro se aplica. La técnica recibe el nombre de parámetro nulo y es una de las más utilizadas en los informes empresariales.

```
<queryString language="sql">
  <![CDATA[
    SELECT titulo, precio, paginas
    FROM libros
    WHERE ($P{categoria} IS NULL OR categoria = $P{categoria})
  ]]>
</queryString>
```

Línea 5

WHERE (\$P{categoria} IS NULL OR categoria = \$P{categoria}) → filtro opcional. Si el parámetro categoria es nulo, la primera condición es verdadera y el filtro no se aplica. Si el parámetro tiene un valor, la segunda condición filtra las filas que coinciden con ese valor.

La técnica funciona porque cada aparición de \$P{categoria} se convierte en un parámetro enlazado del PreparedStatement. Si Java proporciona null, JDBC enlaza SQL NULL: la primera comparación ? IS NULL resulta verdadera y desactiva el filtro; la segunda 1. categoria = ? no necesita ser verdadera. No existe una concatenación textual del valor dentro del SQL.

COMPORTAMIENTO DEL FILTRO OPCIONAL

```
Parámetro = null:
  WHERE (null IS NULL OR categoria = null)
  → (TRUE OR ...) → TRUE
  → Todas las filas se recuperan.

Parámetro = 'Novela':
  WHERE ('Novela' IS NULL OR categoria = 'Novela')
  → (FALSE OR categoria = 'Novela')
  → Solo las filas con categoria = 'Novela'.

Parámetro = '' (cadena vacía):
  WHERE ('' IS NULL OR categoria = '')
  → (FALSE OR categoria = '')
  → Solo las filas con categoria = ''.
```

Qué representa el diagrama: el comportamiento del filtro opcional con tres valores del parámetro. El valor nulo desactiva el filtro. El valor concreto lo activa.

Por qué es relevante: permite construir informes que se adaptan a las preferencias del usuario sin necesidad de definir múltiples consultas.

Bloque 3 — Filtros con múltiples parámetros combinados

Los filtros pueden combinar varios parámetros con operadores lógicos. La combinación de filtros opcionales con AND permite construir consultas que se aplican de forma acumulativa. La combinación con OR permite construir consultas que se aplican de forma alternativa. La elección del operador determina el comportamiento del filtro cuando varios parámetros tienen valor. La combinación de filtros opcionales con AND es la más habitual en los informes empresariales porque permite al usuario ir refinando el conjunto de datos.

```
<queryString language="sql">
  <![CDATA[
    SELECT titulo, precio, paginas, categoria
    FROM libros
    WHERE ($P{categoria} IS NULL OR categoria = $P{categoria})
      AND ($P{precioMinimo} IS NULL OR precio >= $P{precioMinimo})
      AND ($P{precioMaximo} IS NULL OR precio <= $P{precioMaximo})
  ]]>
</queryString>
```

Línea 5

WHERE (\$P{categoria} IS NULL OR categoria = \$P{categoria}) → primer filtro opcional por categoría.

Línea 6	AND (\$P{precioMinimo} IS NULL OR precio >= \$P{precioMinimo}) → segundo filtro opcional por precio mínimo.
Línea 7	AND (\$P{precioMaximo} IS NULL OR precio <= \$P{precioMaximo}) → tercer filtro opcional por precio máximo.

Los filtros combinados con AND se aplican todos a la vez. Si los tres parámetros tienen valor, la consulta devuelve los libros que cumplen las tres condiciones. Si solo uno tiene valor, la consulta devuelve los libros que cumplen esa condición. Si ninguno tiene valor, la consulta devuelve todos los libros. Este comportamiento acumulativo es el que permite al usuario refinar progresivamente el conjunto de datos. La combinación con OR produciría un comportamiento distinto: la consulta devolvería los libros que cumplen al menos una de las condiciones. La elección del operador depende del efecto que se quiera conseguir.

FILTROS COMBINADOS CON AND

Caso 1: todos los parámetros son nulos
→ Todos los libros se recuperan.

Caso 2: solo categoría tiene valor
→ Los libros de esa categoría.

Caso 3: categoría y precio mínimo tienen valor
→ Los libros de esa categoría con precio >= precio mínimo.

Caso 4: los tres parámetros tienen valor
→ Los libros que cumplen las tres condiciones.

El comportamiento es acumulativo.

Qué representa el diagrama: el comportamiento acumulativo de los filtros combinados con AND. Cada parámetro con valor añade una condición adicional.

Por qué es relevante: permite construir informes que se adaptan a distintos niveles de detalle sin modificar la plantilla.

Bloque 4 — Filtrado con printWhenExpression

La propiedad `printWhenExpression` de un elemento o de una banda determina si el elemento se imprime o se omite. La expresión se evalúa en el momento de la emisión y si devuelve `true` el elemento se imprime, si devuelve `false` se omite. Esta propiedad permite aplicar filtros en la plantilla que no pueden expresarse en SQL. Un filtro típico consiste en ocultar las filas cuyo valor de un campo no cumpla una condición. La diferencia con el filtrado en SQL es que el motor ya ha recibido los datos y simplemente decide no imprimirlos.

```
<band height="20">
  <printWhenExpression><![CDATA[$F{precio} > $P{precioMinimo}]]></printWhenExpression>
  ...
</band>
```

Línea 1	<code><band height="20"></code> → declara la banda con su altura.
Línea 2	<code><printWhenExpression><![CDATA[\$F{precio} > \$P{precioMinimo}]]></printWhenExpression></code> → expresión que determina si la banda se imprime. Solo se imprimen las filas cuyo precio supere el mínimo.

La propiedad `printWhenExpression` puede aplicarse a la banda completa o a elementos individuales. Cuando se aplica a la banda, todos los elementos de la banda se imprimen o se omiten juntos. Cuando se aplica a un elemento, solo ese elemento se ve afectado. La combinación de ambas permite construir efectos complejos, como ocultar una columna cuando no hay datos o mostrar un mensaje solo en determinadas condiciones. La propiedad se declara como primer elemento hijo de la banda o del elemento al que se aplica.

APLICACIÓN DE `printWhenExpression`

A la banda completa:

```
<band height="20">
  <printWhenExpression>...</printWhenExpression>
  ...
</band>
```

→ Toda la banda se imprime o se omite.

A un elemento individual:

```
<textField>
  <reportElement .../>
  <printWhenExpression>...</printWhenExpression>
  ...
</textField>
```

→ Solo ese elemento se imprime o se omite.

Qué representa el diagrama: la aplicación de `printWhenExpression` a una banda o a un elemento individual. La elección determina el alcance del filtro.

Por qué es relevante: permite aplicar filtros con distinto nivel de granularidad según el efecto deseado.

Bloque 5 — Combinación de filtros SQL y filtros de plantilla

Un informe profesional combina filtros en SQL y filtros en la plantilla. Los filtros en SQL se aplican a los datos que provienen de la base de datos y reducen el volumen. Los filtros en la plantilla se aplican a los datos que ya están en memoria y permiten condiciones que dependen del contexto. La combinación permite construir informes que son eficientes en el uso de la base de datos y flexibles en la presentación. La división habitual es: filtros de selección en SQL, filtros de presentación en la plantilla.

DISTRIBUCIÓN HABITUAL DE FILTROS

En SQL:

- Filtros por rango de fechas.
- Filtros por categoría.
- Filtros por precio.
- Filtros por disponibilidad.

En la plantilla:

- Ocultar columnas según el parámetro `mostrarDetalle`.
- Mostrar u ocultar filas según el resultado de un cálculo.
- Aplicar estilos condicionales según el valor del parámetro.
- Mostrar mensajes según el estado del informe.

Qué representa el diagrama: la distribución habitual de los filtros entre SQL y la plantilla. Los filtros de selección se aplican en SQL. Los filtros de presentación se aplican en la plantilla.

Por qué es relevante: permite organizar los filtros de forma coherente y aprovechar las ventajas de cada nivel.

La combinación de filtros requiere coordinar los parámetros entre las dos capas. Un mismo parámetro puede utilizarse en la consulta SQL y en la plantilla. Un parámetro `mostrarDetalle` que controla la visibilidad de una columna en la plantilla no se utiliza en la consulta. Un parámetro `categoria` que filtra los registros en la consulta se utiliza también en la plantilla para mostrar el valor en el encabezado. La coherencia entre las dos capas es la que permite construir un informe que se comporte de forma predecible. La documentación de los parámetros y de su uso en cada capa es una buena práctica.

COORDINACIÓN DE PARÁMETROS ENTRE CAPAS

Parámetro `categoria`:

- Usado en SQL: `WHERE categoria = ${categoria}`
- Usado en la plantilla: cabecera "Categoría: " + `${categoria}`

Parámetro `mostrarDetalle`:

- Usado en SQL: no se usa
- Usado en la plantilla: `printWhenExpression` en la columna

Parámetro `precioMinimo`:

- Usado en SQL: `WHERE precio >= ${precioMinimo}`
- Usado en la plantilla: puede mostrarse en la cabecera

Qué representa el diagrama: la coordinación de los parámetros entre la capa SQL y la capa de plantilla. Algunos parámetros se usan en ambas capas, otros solo en una.

Por qué es relevante: permite planificar el uso de cada parámetro y documentar su propósito en cada capa.

Resumen rápido de la teoría

- El filtrado puede aplicarse en la consulta SQL o en la plantilla.
- El filtrado en SQL es más eficiente pero limitado a las condiciones que el motor de base de datos puede evaluar.
- El filtrado en la plantilla es más flexible pero recibe todos los datos.
- El filtro opcional con parámetro nulo se aplica solo cuando el parámetro tiene valor.
- La combinación de filtros con `AND` es acumulativa.
- La combinación de filtros con `OR` es alternativa.
- La propiedad `printWhenExpression` controla la visibilidad de una banda o de un elemento.
- La distribución habitual es filtros de selección en SQL y filtros de presentación en la plantilla.

Punto 4.3 — Variables

Módulo, proyecto y objetivos de aprendizaje

Módulo: 4 — Parámetros y lógica (3 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial

Punto: 4.3 — Variables

Objetivos de aprendizaje

- Comprender el ciclo de vida de una variable a lo largo del llenado.
- Elegir el tipo de cálculo adecuado según el valor que se quiere acumular.
- Elegir el tipo de reinicio adecuado según el alcance del valor.
- Declarar variables que combinan campos, parámetros y otras variables.
- Utilizar variables en bandas específicas para construir subtotales y totales.
- Documentar las variables del proyecto EditorialReports.

Parte teórica

Bloque 1 — Ciclo de vida de una variable

Una variable es un valor que el motor calcula a lo largo del llenado según su expresión y su tipo de cálculo. El ciclo de vida de una variable comienza cuando el motor la inicializa al valor inicial de su tipo. Continúa con la evaluación de la expresión en cada registro y la actualización del valor acumulado según el tipo de cálculo. Termina cuando el motor reinicia la variable según su tipo de reinicio o cuando el llenado finaliza. La variable no existe antes del llenado ni después de él: es un valor que se construye progresivamente a lo largo del proceso. Esta característica la distingue del campo, que se resuelve en cada registro, y del parámetro, que se resuelve al inicio.

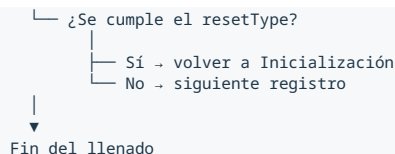
```
<variable name="AcumuladoImporte" class="java.lang.Double" calculation="Sum" resetType="Report">
  <variableExpression><![CDATA[${importe_total}]]></variableExpression>
</variable>
```

Línea 1	<code><variable name="AcumuladoImporte" class="java.lang.Double" calculation="Sum" resetType="Report"></code> → declara una variable de tipo Double que se calcula mediante suma y se reinicia al inicio del informe.
Línea 2	<code><variableExpression><![CDATA[\${importe_total}]]></variableExpression></code> → expresión que se evalúa en cada registro y cuyo valor se acumula según el cálculo.
Línea 3	<code></variable></code> → cierra la declaración de la variable.

El ciclo de vida de una variable tiene cuatro fases. La primera es la inicialización: si existe `initialValueExpression`, el motor la evalúa al reiniciar la variable; en caso contrario, el valor inicial depende del cálculo y de su incrementador, por lo que no debe asumirse de forma general que todos los números comienzan en 0 o las cadenas en vacío. La segunda es la evaluación: en cada registro, el motor evalúa la expresión y obtiene un valor. La tercera es la acumulación: el motor aplica el tipo de cálculo al valor actual y al valor anterior para obtener el nuevo valor acumulado. La cuarta es el reinicio: cuando se cumple la condición del tipo de reinicio, el motor vuelve a la fase de inicialización. Las cuatro fases se repiten a lo largo del llenado según el tipo de reinicio.

CICLO DE VIDA DE UNA VARIABLE

```
Inicio del llenado
|
↓
Inicialización:
  AcumuladoImporte = 0.0
|
↓
Para cada registro:
  |
  |— Evaluación:
  |   valor = ${importe_total}
  |
  |— Acumulación:
  |   AcumuladoImporte = AcumuladoImporte + valor
  |
  ↓
```



Qué representa el diagrama: el ciclo de vida de una variable desde la inicialización hasta el final del llenado. La acumulación se realiza registro a registro según el tipo de cálculo.

Por qué es relevante: permite comprender por qué una variable muestra un valor distinto según la banda en la que se consulte. En la banda Detail muestra el valor acumulado hasta el registro actual. En la banda Summary muestra el valor final.

Bloque 2 — Tipos de cálculo

El atributo `calculation` determina la operación que el motor aplica en cada evaluación. Los tipos de cálculo se agrupan en cuatro categorías. La primera es la categoría de agregación aritmética: `Sum` acumula la suma de los valores, `Average` acumula la suma y el contador para calcular la media al final, `StandardDeviation` acumula los valores necesarios para calcular la desviación estándar, `Variance` acumula los valores necesarios para calcular la varianza. La segunda es la categoría de agregación lógica: `Count` cuenta los valores no nulos, `DistinctCount` cuenta los valores distintos. La tercera es la categoría de extremos: `Lowest` mantiene el valor mínimo, `Highest` mantiene el valor máximo. La cuarta es la categoría de posición: `First` mantiene el primer valor evaluado, `Nothing` mantiene el último valor evaluado.

```

<variable name="PrecioMedio" class="java.lang.Double" calculation="Average" resetType="Report">
  <variableExpression><![CDATA[{precio_medio}]]></variableExpression>
</variable>
<variable name="PrecioMaximo" class="java.lang.Double" calculation="Highest" resetType="Report">
  <variableExpression><![CDATA[{precio_medio}]]></variableExpression>
</variable>
<variable name="NumeroLibros" class="java.lang.Integer" calculation="Count" resetType="Report">
  <variableExpression><![CDATA[{titulo}]]></variableExpression>
</variable>

```

Línea 1-3	<code><variable name="PrecioMedio" ...></code> → declara una variable que calcula la media de los precios medios.
Línea 4-6	<code><variable name="PrecioMaximo" ...></code> → declara una variable que mantiene el precio máximo.
Línea 7-9	<code><variable name="NumeroLibros" ...></code> → declara una variable que cuenta los títulos no nulos.

TIPOS DE CÁLCULO

Agregación aritmética:

- `Sum` → Suma de los valores
- `Average` → Media de los valores
- `StandardDeviation` → Desviación estándar
- `Variance` → Varianza

Agregación lógica:

- `Count` → Cuenta los valores no nulos
- `DistinctCount` → Cuenta los valores distintos

Extremos:

- `Lowest` → Valor mínimo
- `Highest` → Valor máximo

Posición:

- `First` → Primer valor evaluado
- `Nothing` → Último valor evaluado (sin acumulación)

Qué representa el diagrama: los tipos de cálculo agrupados en cuatro categorías. Cada categoría resuelve un tipo distinto de valor.

Por qué es relevante: permite elegir el tipo de cálculo adecuado según el valor que se quiera calcular. La elección incorrecta produce valores incorrectos sin error de compilación.

Bloque 3 — Tipos de reinicio

El atributo `resetType` determina cuándo el motor reinicia la variable a su valor inicial. Los tipos de reinicio se agrupan en cinco categorías. El reinicio `Report` se aplica al inicio del informe, una sola vez. El reinicio `Page` se aplica al inicio de cada página. El reinicio `Column` se aplica al inicio de cada columna. El reinicio `Group` se aplica al inicio de cada grupo de la agrupación indicada. El reinicio `None` no se aplica nunca: la variable acumula a lo largo de todo el llenado sin reiniciarse. La combinación del tipo de cálculo y del tipo de reinicio determina el valor final de la variable.

```
<variable name="TotalPagina" class="java.lang.Double" calculation="Sum" resetType="Page">
  <variableExpression><![CDATA[$F{importe_total}]]></variableExpression>
</variable>
<variable name="TotalInforme" class="java.lang.Double" calculation="Sum" resetType="Report">
  <variableExpression><![CDATA[$F{importe_total}]]></variableExpression>
</variable>
```

Línea 1-3

`<variable name="TotalPagina" ...>` → declara una variable que acumula el importe total de cada página. Se reinicia al inicio de cada página.

Línea 4-6

`<variable name="TotalInforme" ...>` → declara una variable que acumula el importe total del informe. Se reinicia al inicio del informe.

TIPOS DE REINICIO

Report	→ Al inicio del informe (una sola vez)
Page	→ Al inicio de cada página
Column	→ Al inicio de cada columna
Group	→ Al inicio de cada grupo de la agrupación indicada
None	→ Sin reinicio (acumula a lo largo de todo el llenado)

Qué representa el diagrama: los tipos de reinicio disponibles. El tipo determina el alcance del valor acumulado.

Por qué es relevante: permite construir subtotales por página, por grupo o por informe. La elección del tipo de reinicio determina la granularidad del valor.

Bloque 4 — Variables que combinan campos, parámetros y otras variables

Una variable puede combinar campos, parámetros y otras variables en su expresión. La combinación permite construir valores derivados que dependen de varias fuentes. Una variable puede multiplicar un campo por un parámetro para calcular un importe con IVA. Puede dividir un acumulado entre un parámetro que representa el número de elementos. Puede combinar el valor de otra variable con un parámetro para calcular un porcentaje. La expresión de una variable tiene acceso al mismo contexto que las expresiones de los campos: campos, parámetros y variables declaradas antes de ella.

```
<parameter name="tipoIva" class="java.lang.Double"/>
<variable name="ImporteConIva" class="java.lang.Double" calculation="Sum" resetType="Report">
  <variableExpression><![CDATA[$F{importe_total} * (1 + $P{tipoIva})]]></variableExpression>
</variable>
<variable name="PorcentajeSobreTotal" class="java.lang.Double" calculation="Nothing" resetType="Report">
  <variableExpression><![CDATA[$V{ImporteConIva} / $V{importe_total}]]></variableExpression>
</variable>
```

Línea 1

`<parameter name="tipoIva" class="java.lang.Double"/>` → declara el parámetro tipoIva.

Línea 2-4

`<variable name="ImporteConIva" ...>` → declara una variable que acumula el importe total multiplicado por $(1 + \text{tipoIva})$.

Línea 5-7

`<variable name="PorcentajeSobreTotal" ...>` → declara una variable que combina dos veces la misma variable. La expresión es un ejemplo didáctico; en la práctica se combinaría con otras variables para calcular porcentajes.

Las variables pueden referenciar otras variables declaradas antes. El orden de declaración determina la disponibilidad: una variable solo puede referenciar variables declaradas antes que ella. Si se declara una variable que referencia una variable declarada después, el compilador lanza un error de resolución. La convención es declarar las variables en el orden en que se necesitan: primero las que dependen solo de campos y parámetros, después las que dependen de las anteriores. La organización en cascada permite construir valores cada vez más derivados sin perder la legibilidad.

VARIABLES EN CASCADA

```
Nivel 1: variables que dependen de campos y parámetros.
  ImporteConIva = SUM($F{importe_total} * (1 + $P{tipoIva}))

Nivel 2: variables que dependen de variables del nivel 1.
  PorcentajeIva = $V{ImporteConIva} - $V{TotalImporte}

Nivel 3: variables que dependen de variables del nivel 2.
  RatioIva = $V{PorcentajeIva} / $V{TotalImporte}
```

Qué representa el diagrama: la organización en cascada de las variables. Cada nivel depende de los niveles anteriores.

Por qué es relevante: permite construir valores derivados complejos sin perder la legibilidad y sin errores de resolución.

Bloque 5 — Variables en bandas específicas

El valor que muestra una variable depende de la banda en la que se consulte. En la banda Detail, la variable muestra el valor acumulado hasta el registro actual. En la banda Page Footer, muestra el valor acumulado en la página actual. En la banda Summary, muestra el valor final del informe. La elección de la banda determina el valor que el lector ve. Una misma variable puede consultarse en varias bandas y mostrar valores distintos en cada una. Esta característica es la que permite construir informes con subtotales por página y totales generales a partir de una única declaración.

```
<variable name="TotalImporte" class="java.lang.Double" calculation="Sum" resetType="Report">
  <variableExpression><![CDATA[{$F{importe_total}}]></variableExpression>
</variable>
```

Línea 1-3

la variable TotalImporte con cálculo Sum y reinicio Report. Se acumula a lo largo de todo el informe.

VALOR DE LA VARIABLE SEGÚN LA BANDA

Banda Detail:

En cada registro muestra el total acumulado hasta ese momento.

Registro 1: 159.60

Registro 2: 294.60

Registro 3: 411.60

...

Banda Page Footer:

En cada página muestra el total acumulado en esa página.

Página 1: 633.40 (todos los registros caben en una página)

Banda Summary:

Muestra el total final del informe.

Total: 633.40

Qué representa el diagrama: el valor de la variable según la banda. En la banda Detail muestra el valor acumulado hasta el registro actual. En la banda Summary muestra el valor final.

Por qué es relevante: permite decidir en qué banda colocar la variable según el valor que se quiera mostrar. La elección de la banda es tan importante como la elección del cálculo y del reinicio.

La combinación de variables con distinto reinicio permite construir informes con múltiples niveles de agregación. Una variable con reinicio Page y cálculo Sum muestra el subtotal de cada página. Una variable con reinicio Report y cálculo Sum muestra el total general. La diferencia entre el total general y la suma de los subtotales de página es cero. Una variable con reinicio Group muestra el subtotal de cada grupo. La diferencia entre el total general y la suma de los subtotales de grupo también es cero. La coherencia entre los niveles de agregación es una de las características que hacen que JasperReports sea adecuado para informes financieros.

MÚLTIPLES NIVELES DE AGREGACIÓN

Variable TotalPagina (Sum, Page):

Página 1: 320.00

Página 2: 328.40

Variable TotalInforme (Sum, Report):

Total: 633.40

Comprobación: 320.00 + 328.40 = 633.40 ✓

Qué representa el diagrama: la coherencia entre los niveles de agregación. La suma de los subtotales de página es igual al total general.

Por qué es relevante: permite verificar la corrección de las variables comparando los niveles de agregación.

Resumen rápido de la teoría

- Una variable se calcula a lo largo del llenado según su expresión y su tipo de cálculo.
- El ciclo de vida tiene cuatro fases: inicialización, evaluación, acumulación y reinicio.
- Los tipos de cálculo se agrupan en agregación aritmética, agregación lógica, extremos y posición.
- Los tipos de reinicio son Report, Page, Column, Group y None.
- Las variables pueden combinar campos, parámetros y otras variables.
- Las variables se organizan en cascada para construir valores derivados.

- El valor de una variable depende de la banda en la que se consulte.
 - La coherencia entre niveles de agregación permite verificar la corrección de las variables.
-
-

Punto 4.4 — Expresiones avanzadas

Módulo, proyecto y objetivos de aprendizaje

Módulo: 4 — Parámetros y lógica (3 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial

Punto: 4.4 — Expresiones avanzadas

Objetivos de aprendizaje

- Construir expresiones con operadores ternarios anidados.
- Utilizar métodos avanzados de `String`, `Date` y `Double` en expresiones.
- Invocar métodos estáticos de clases de utilidad con nombre completamente cualificado.
- Combinar campos, parámetros y variables en una misma expresión compleja.
- Depurar expresiones avanzadas con las herramientas del entorno.
- Documentar las expresiones avanzadas del proyecto EditorialReports.

Parte teórica

Bloque 1 — Operadores ternarios anidados

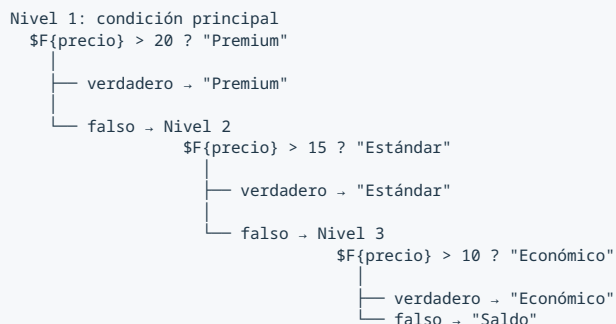
Un operador ternario anidado es una expresión que contiene otro operador ternario en su rama verdadera o en su rama falsa. La anidación permite construir clasificaciones de tres o más niveles sin necesidad de escribir condicionales complejos. La sintaxis es `condición1 ? resultado1 : (condición2 ? resultado2 : resultado3)`. Los paréntesis alrededor del ternario interno son obligatorios para que el compilador interprete la expresión en el orden correcto. Sin los paréntesis, el compilador asocia el `:` con el primer `?` y produce un resultado distinto al esperado.

```
<textFieldExpression><![CDATA[{$F{precio} > 20 ? "Premium" : ($F{precio} > 15 ? "Estándar" : "Económico")}]></textFieldExpression>
```

Línea 1	<code><textFieldExpression><![CDATA[...]]></textFieldExpression></code> → abre y cierra el bloque de la expresión.
Línea 1 (continuación)	<code>{\$F{precio} > 20 ? "Premium"}</code> → primera condición. Si el precio es superior a 20, la expresión devuelve "Premium".
Línea 1 (continuación)	<code>: (\$F{precio} > 15 ? "Estándar" : "Económico")</code> → rama falsa. Contiene un segundo operador ternario que evalúa si el precio es superior a 15. Si lo es, devuelve "Estándar". Si no, devuelve "Económico". Los paréntesis alrededor del ternario interno son obligatorios.

La anidación puede extenderse a cuatro, cinco o más niveles, aunque a partir del cuarto nivel la expresión se vuelve difícil de leer. La práctica recomendada consiste en extraer la lógica compleja a una variable o a un método estático cuando la clasificación supera los tres niveles. Un método estático en una clase de utilidad permite encapsular la lógica y reutilizarla en varias expresiones del informe. La expresión del JXML se limita a invocar el método y presentar el resultado. Esta separación entre la lógica y la presentación es la que mantiene la plantilla legible a largo plazo.

OPERADOR TERNARIO ANIDADO



Qué representa el diagrama: la estructura de un operador ternario anidado de cuatro niveles. Cada nivel se evalúa solo si el anterior ha resultado falso.

Por qué es relevante: permite construir clasificaciones complejas con una única expresión sin escribir bloques de código.

Bloque 2 — Métodos avanzados de String

La clase String de Java ofrece un conjunto de métodos que resultan útiles en las expresiones de JasperReports. El método `substring(inicio, fin)` devuelve una subcadena entre dos posiciones. El método `indexOf(cadena)` devuelve la posición de la primera aparición de una subcadena. El método `lastIndexOf(cadena)` devuelve la posición de la última aparición. El método `replace(antiguo, nuevo)` reemplaza todas las apariciones de una subcadena por otra. El método `replaceAll(regex, nuevo)` reemplaza las coincidencias de una expresión regular. El método `trim()` elimina los espacios al principio y al final. El método `split(separador)` divide la cadena en un arreglo de subcadenas.

```
<textFieldExpression><![CDATA[ $\text{\$F\{titulo\}.length() > 30 ? \text{\$F\{titulo\}.substring(0, 27).trim() + "...": \text{\$F\{titulo\}}$ ]]></textFieldExpression>
```

Línea 1	<code><textFieldExpression><![CDATA[...]]></textFieldExpression></code> → abre y cierra el bloque de la expresión.
Línea 1 (continuación)	<code>$\text{\\$F\{titulo\}.length() > 30}$</code> → condición. Comprueba si el título tiene más de 30 caracteres.
Línea 1 (continuación)	<code>$\text{\\$F\{titulo\}.substring(0, 27).trim() + "..."$</code> → si la condición es verdadera, extrae los primeros 27 caracteres, elimina los espacios al final y añade puntos suspensivos.
Línea 1 (continuación)	<code>$:\text{\\$F\{titulo\}}$</code> → si la condición es falsa, devuelve el título completo.

El método `replace` permite construir expresiones que normalizan los valores antes de imprimirlos. Un título con comillas dobles puede transformarse en un título con comillas simples. Un código con guiones puede transformarse en un código con puntos. El método `split` permite dividir una cadena en varias partes y acceder a cada una por su índice. Una cadena "Apellido, Nombre" puede dividirse por la coma y reconstruirse como "Nombre Apellido". La combinación de estos métodos permite construir expresiones que transforman los datos sin necesidad de procesarlos previamente en el programa Java.

MÉTODOS AVANZADOS DE STRING

```
longitud:       $\text{\$F\{titulo\}.length()}$ 
subcadena:     $\text{\$F\{titulo\}.substring(0, 10)}$ 
primera pos:   $\text{\$F\{titulo\}.indexOf("a")}$ 
última pos:    $\text{\$F\{titulo\}.lastIndexOf("a")}$ 
reemplazar:    $\text{\$F\{titulo\}.replace("á", "a")}$ 
reemplazar re:  $\text{\$F\{titulo\}.replaceAll("[aeiou]", "")}$ 
recortar:      $\text{\$F\{titulo\}.trim()}$ 
dividir:       $\text{\$F\{titulo\}.split(",")}$ 
mayúsculas:    $\text{\$F\{titulo\}.toUpperCase()}$ 
minúsculas:    $\text{\$F\{titulo\}.toLowerCase()}$ 
contiene:      $\text{\$F\{titulo\}.contains("sol")}$ 
empieza con:   $\text{\$F\{titulo\}.startsWith("Cien")}$ 
termina con:   $\text{\$F\{titulo\}.endsWith("edad")}$ 
```

Qué representa el diagrama: los métodos avanzados de String disponibles en las expresiones. Cada método resuelve un tipo de transformación.

Por qué es relevante: permite transformar los datos en el momento de la impresión sin necesidad de procesarlos previamente.

Bloque 3 — Métodos avanzados de Date y Double

La clase Date de Java ofrece métodos para comparar fechas y calcular diferencias. El método `before(fecha)` devuelve true si la fecha es anterior al argumento. El método `after(fecha)` devuelve true si es posterior. El método `compareTo(fecha)` devuelve un entero negativo, cero o positivo según el orden. El método `getTime()` devuelve el número de milisegundos desde el 1 de enero de 1970. La diferencia entre dos `getTime` dividida entre el número de milisegundos de un día produce el número de días entre dos fechas. La clase Double de Java ofrece métodos para redondear y comprobar valores. El método `intValue()` convierte a entero truncando los decimales. El método `isNaN()` comprueba si el valor es un número válido.

```
<textFieldExpression><![CDATA[
    (int) ((new java.util.Date().getTime() -  $\text{\$F\{fechaPublicacion\}.getTime()}$ ) / (1000L * 60 * 60 * 24))
]]></textFieldExpression>
```

Línea 2	<code>(int)</code> → convierte el resultado de tipo long a int para que el campo pueda declararse como <code>java.lang.Integer</code> .
----------------	---

Línea 2 (continuación)	<code>new java.util.Date().getTime()</code> → devuelve los milisegundos de la fecha actual.
Línea 2 (continuación)	<code>- \${fechaPublicacion}.getTime()</code> → resta los milisegundos de la fecha de publicación.
Línea 2 (continuación)	<code>/ (1000L * 60 * 60 * 24)</code> → divide entre el número de milisegundos de un día. El sufijo L indica que el número es de tipo long para evitar el desbordamiento.

Los métodos de `Double` permiten construir expresiones que redondean los valores antes de imprimirlos. El método `Math.round(valor)` devuelve el entero más próximo. El método `Math.floor(valor)` devuelve el entero inferior. El método `Math.ceil(valor)` devuelve el entero superior. La clase `java.text.DecimalFormat` permite aplicar un formato específico a un valor. La combinación de estos métodos permite construir expresiones que presentan los datos con el grado de precisión adecuado sin necesidad de recurrir a patrones del elemento `textField`.

MÉTODOS AVANZADOS DE DATE Y DOUBLE

```
Date:
    ${fecha}.before(${P{fechaCorte}})
    ${fecha}.after(${P{fechaCorte}})
    ${fecha}.compareTo(${P{fechaCorte}})
    ${fecha}.getTime()
    new java.text.SimpleDateFormat("dd/MM/yyyy").format(${fecha})

Double:
    Math.round(${F{precio}})
    Math.floor(${F{precio}})
    Math.ceil(${F{precio}})
    new java.text.DecimalFormat("#,##0.00").format(${F{precio}})
    ${F{precio}}.intValue()
    ${F{precio}}.isNaN()
```

Qué representa el diagrama: los métodos avanzados de `Date` y `Double` disponibles en las expresiones. Cada uno resuelve un tipo de cálculo.

Por qué es relevante: permite construir expresiones que calculan diferencias, redondean valores y aplican formatos sin necesidad de modificar los datos.

Bloque 4 — Métodos estáticos de clases de utilidad

Las expresiones de JasperReports pueden invocar métodos estáticos de cualquier clase Java que esté en el classpath. El nombre completamente cualificado de la clase evita la necesidad de importarla en el JRXML. Los métodos estáticos más utilizados son los de las clases `Math`, `Integer`, `Double`, `String` y `java.text.SimpleDateFormat`. La clase `Math` ofrece métodos para cálculos matemáticos como `abs`, `max`, `min`, `pow` y `sqrt`. La clase `Integer` ofrece métodos para convertir cadenas a enteros como `parseInt`. La clase `Double` ofrece métodos para convertir cadenas a decimales como `parseDouble`. La clase `String` ofrece métodos para construir cadenas como `format` y `valueOf`.

```
<textFieldExpression><![CDATA[
    Math.max(${F{unidades_vendidas}}, Math.max(${V{TotalUnidades}} / 2, 10))
]]></textFieldExpression>
```

Línea 2	<code>Math.max(\${F{unidades_vendidas}}, ...)</code> → invoca el método estático <code>max</code> de la clase <code>Math</code> con dos argumentos.
Línea 2 (continuación)	<code>Math.max(\${V{TotalUnidades}} / 2, 10)</code> → segundo argumento. Es a su vez una llamada a <code>Math.max</code> que devuelve el mayor entre la mitad del total de unidades y 10.
Línea 2 (continuación)	<code>)</code> → cierra la llamada externa. El resultado es el mayor de los tres valores.

Los métodos estáticos también pueden invocarse sobre clases definidas por el propio proyecto. Una clase `EditorialUtils` con métodos estáticos para calcular descuentos, formatear códigos o validar valores puede utilizarse en las expresiones del informe. La clase debe estar en el classpath y el método debe ser `public static`. Esta técnica permite encapsular la lógica de negocio en una clase Java y reutilizarla en varios informes. La expresión del JRXML se limita a invocar el método, lo que mantiene la plantilla legible y facilita las pruebas unitarias de la lógica.

MÉTODOS ESTÁTICOS FRECUENTES

```
Math:
    Math.abs(x)
    Math.max(a, b)
```

```

Math.min(a, b)
Math.pow(base, exp)
Math.sqrt(x)
Math.round(x)

Integer / Double:
Integer.parseInt("42")
Double.parseDouble("19.95")
Integer.valueOf("42")
Double.valueOf("19.95")

String:
String.format("%s - %s", ${titulo}, ${autor})
String.valueOf(${precio})
String.join(" ", new String[]{"a", "b", "c"})

SimpleDateFormat:
new java.text.SimpleDateFormat("dd/MM/yyyy").format(${fecha})
new java.text.SimpleDateFormat("yyyy-MM-dd").parse("2026-09-22")

```

Qué representa el diagrama: los métodos estáticos más utilizados en las expresiones. Cada uno resuelve un tipo de operación.

Por qué es relevante: permite construir expresiones que realizan cálculos matemáticos, conversiones de tipo y formateos sin necesidad de escribir código Java adicional.

Bloque 5 — Combinación de campos, parámetros y variables en expresiones complejas

Las expresiones avanzadas combinan campos, parámetros y variables en una misma instrucción. La combinación permite construir valores que dependen simultáneamente de los datos del registro actual, de las instrucciones del usuario y del estado del informe. Una expresión puede multiplicar un campo por un parámetro, dividir el resultado entre una variable y aplicar un método estático para redondear. La expresión completa se evalúa en el momento de la emisión y el resultado se imprime en el documento. La combinación de los tres tipos de referencias es la que hace que un informe profesional sea realmente dinámico.

```

<textFieldExpression><![CDATA[
    Math.round(${TotalImporte} / ${numElementos} * (1 + ${tipoIva}) * 100.0) / 100.0
]]></textFieldExpression>

```

Línea 2	<code>Math.round(...)</code> → invoca el método estático <code>round</code> de la clase <code>Math</code> para redondear el resultado.
Línea 2 (continuación)	<code>\${V{TotalImporte}} / \${P{numElementos}}</code> → divide la variable <code>TotalImporte</code> entre el parámetro <code>numElementos</code> . El resultado es la media.
Línea 2 (continuación)	<code>* (1 + \${P{tipoIva}})</code> → multiplica por el factor de IVA.
Línea 2 (continuación)	<code>* 100.0) / 100.0</code> → multiplica por 100, redondea y divide entre 100 para obtener dos decimales. Esta técnica es habitual para redondear a dos decimales sin utilizar <code>DecimalFormat</code> .

La combinación de campos, parámetros y variables requiere atención al orden de evaluación y a los tipos. El motor evalúa las expresiones de izquierda a derecha según la precedencia de los operadores. Los paréntesis modifican la precedencia. Los tipos de las variables y de los parámetros deben ser compatibles con las operaciones que se realizan. Una división entre enteros produce un entero truncado, mientras que una división entre decimales produce un decimal. La conversión de tipos puede realizarse con los métodos `intValue`, `doubleValue` o con los métodos estáticos `parseInt` y `parseDouble`. La coherencia de tipos es la primera línea de defensa contra los errores de evaluación.

COMBINACIÓN DE LOS TRES TIPOS DE REFERENCIAS

```

Ejemplo 1: media con IVA redondeada
Math.round(${V{TotalImporte}} / ${P{numElementos}} * (1 + ${P{tipoIva}}) * 100.0) / 100.0

Ejemplo 2: clasificación con umbral
${F{unidades_vendidas}} > ${P{umbral}} ? "Alta rotación" : "Baja rotación"

Ejemplo 3: porcentaje sobre total
${F{importe_total}} / ${V{TotalImporte}} * 100.0

Ejemplo 4: fecha formateada con patrón dinámico
new SimpleDateFormat(${P{formatoFecha}}).format(${P{fechaInforme}})

Ejemplo 5: título recortado con indicador
${F{titulo}}.length() > ${P{longitudMaxima}} ?
    ${F{titulo}}.substring(0, ${P{longitudMaxima}}) + "..." :
    ${F{titulo}}

```

Qué representa el diagrama: cinco ejemplos de expresiones que combinan los tres tipos de referencias. Cada uno resuelve un caso de uso distinto.

Por qué es relevante: permite identificar el patrón adecuado para cada necesidad y construir expresiones complejas con garantías.

La depuración de expresiones complejas se realiza con las herramientas habituales. El panel Problems detecta los errores de compilación. La vista Console muestra la traza de las excepciones. La previsualización muestra el resultado con datos reales. La buena práctica consiste en construir la expresión por partes y verificar cada parte antes de combinarla con la siguiente. Una expresión de cinco operaciones se construye en cinco pasos, verificando el resultado después de cada paso. Esta aproximación incremental reduce el tiempo de depuración y facilita la localización de los errores.

CONSTRUCCIÓN INCREMENTAL DE UNA EXPRESIÓN

Paso 1: `{F}{precio}`
→ valor del campo

Paso 2: `{F}{precio} * (1 + {P}{tipoIva})`
→ precio con IVA

Paso 3: `{F}{precio} * (1 + {P}{tipoIva}) * {F}{cantidad}`
→ importe con IVA

Paso 4: `Math.round({F}{precio} * (1 + {P}{tipoIva}) * {F}{cantidad} * 100.0) / 100.0`
→ importe con IVA redondeado a dos decimales

Cada paso se verifica antes de añadir el siguiente.

Qué representa el diagrama: la construcción incremental de una expresión compleja. Cada paso añade una operación y se verifica antes de continuar.

Por qué es relevante: permite abordar las expresiones complejas de forma sistemática y evitar errores acumulados.

Resumen rápido de la teoría

- Los operadores ternarios anidados permiten construir clasificaciones de tres o más niveles.
- Los paréntesis alrededor del ternario interno son obligatorios.
- Los métodos de `String` permiten manipular cadenas en las expresiones.
- Los métodos de `Date` y `Double` permiten comparar fechas y redondear valores.
- Los métodos estáticos de clases de utilidad pueden invocarse con nombre completamente cualificado.
- La combinación de campos, parámetros y variables construye expresiones dinámicas.
- La construcción incremental facilita la depuración de expresiones complejas.
- La coherencia de tipos es la primera línea de defensa contra los errores de evaluación.

Punto 4.5 — Lógica condicional

Módulo, proyecto y objetivos de aprendizaje

Módulo: 4 — Parámetros y lógica (3 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial

Punto: 4.5 — Lógica condicional

Objetivos de aprendizaje

- Aplicar condiciones compuestas con los operadores &&, || y !.
- Configurar `printWhenExpression` en bandas y elementos individuales.
- Definir estilos condicionales con `conditionalStyle` y varias condiciones.
- Combinar condiciones sobre campos, parámetros y variables.
- Aplicar la lógica condicional a la visibilidad de columnas completas.
- Documentar la lógica condicional del proyecto EditorialReports.

Parte teórica

Bloque 1 — Condiciones compuestas con operadores lógicos

Una condición compuesta es una expresión booleana que combina dos o más condiciones simples mediante operadores lógicos. Los operadores disponibles son && para la conjunción, || para la disyunción y ! para la negación. La conjunción && devuelve verdadero solo si ambas condiciones son verdaderas. La disyunción || devuelve verdadero si al menos una de las condiciones es verdadera. La negación ! invierte el valor de la condición. La combinación de estos operadores permite expresar reglas complejas que dependen de varios valores simultáneamente.

```
<printWhenExpression><![CDATA[{$P{mostrarDetalle}.booleanValue() && $F{unidades_vendidas} > 5}]]></printWhenExpression>
```

Línea 1	<code><printWhenExpression><![CDATA[...]]></printWhenExpression></code> → abre y cierra el bloque de la condición de visibilidad.
Línea 1 (continuación)	<code>\$P{mostrarDetalle}.booleanValue()</code> → primera condición. Devuelve verdadero si el parámetro <code>mostrarDetalle</code> es verdadero.
Línea 1 (continuación)	<code>&&</code> → operador de conjunción. La condición completa es verdadera solo si ambas condiciones son verdaderas.
Línea 1 (continuación)	<code>\$F{unidades_vendidas} > 5</code> → segunda condición. Devuelve verdadero si el campo <code>unidades_vendidas</code> es superior a 5.

Los operadores lógicos tienen una precedencia distinta. El operador ! tiene la precedencia más alta, seguido de && y después de ||. Esta precedencia puede modificarse con paréntesis. Sin paréntesis, la expresión `a || b && c` se evalúa como `a || (b && c)`, no como `(a || b) && c`. La práctica recomendada consiste en utilizar paréntesis siempre que la expresión combine más de un operador lógico. Los paréntesis no afectan al rendimiento pero mejoran la legibilidad y eliminan las ambigüedades. La coherencia en el uso de paréntesis facilita la lectura de las expresiones por parte de otros desarrolladores.

PRECEDENCIA DE OPERADORES LÓGICOS

Sin paréntesis:

`a || b && c` → `a || (b && c)`

`!a && b` → `(!a) && b`

`a && b || c` → `(a && b) || c`

Con paréntesis:

`(a || b) && c` → evalúa primero `a || b`

`a && (b || c)` → evalúa primero `b || c`

Qué representa el diagrama: la precedencia de los operadores lógicos y el efecto de los paréntesis. La agrupación con paréntesis modifica el orden de evaluación.

Por qué es relevante: permite escribir condiciones compuestas que se evalúan en el orden correcto y evitar errores sutiles en la lógica.

Bloque 2 — printWhenExpression en bandas y elementos

La propiedad `printWhenExpression` controla la visibilidad de una banda o de un elemento individual. La expresión se evalúa en el momento de la emisión y si devuelve `true` la banda o el elemento se imprime, si devuelve `false` se omite. La propiedad se declara como primer elemento hijo del elemento al que se aplica. En una banda se declara inmediatamente después de la apertura de la banda. En un elemento se declara inmediatamente después del bloque `reportElement`.

```
<band height="20">
  <printWhenExpression><![CDATA[{$F{precio} > 10}]]></printWhenExpression>
  ...
</band>
```

Línea 1 `<band height="20">` → declara la banda con su altura.

Línea 2 `<printWhenExpression><![CDATA[{$F{precio} > 10}]]></printWhenExpression>` → condición de visibilidad. La banda se imprime solo si el precio es superior a 10.

Cuando la propiedad se aplica a un elemento individual, el resto de la banda se imprime con normalidad. Este comportamiento permite ocultar columnas específicas sin afectar a las demás. La combinación de la propiedad en la banda y en los elementos permite construir visibilidades en cascada: la banda se imprime si se cumple una condición general y cada elemento se imprime si se cumple una condición específica. La condición de la banda tiene prioridad sobre las condiciones de los elementos: si la banda no se imprime, ninguno de sus elementos se imprime aunque sus condiciones individuales sean verdaderas.

VISIBILIDAD EN CASCADA

```
Banda Detail:
  printWhenExpression: $P{mostrarDetalle}
  |
  |--- false → la banda NO se imprime (ningún elemento)
  |
  |--- true → la banda se imprime
        |
        |--- Elemento 1: printWhenExpression: {$F{precio} > 10}
              |
              |--- false → el elemento NO se imprime
              |
              |--- true → el elemento se imprime
        |
        |--- Elemento 2: sin condición → se imprime siempre
```

Qué representa el diagrama: la visibilidad en cascada de la banda y de los elementos. La condición de la banda controla todos los elementos. Las condiciones individuales controlan cada elemento por separado.

Por qué es relevante: permite construir visibilidades complejas combinando condiciones generales y específicas.

Bloque 3 — Estilos condicionales con conditionalStyle

Un `conditionalStyle` añade propiedades visuales cuando su condición booleana es verdadera. Un estilo puede contener varias reglas. Si varias condiciones verdaderas modifican **la misma propiedad**, JasperReports conserva el valor de la primera regla verdadera que estableció esa propiedad. Por ello no debe enseñarse la regla simplista “gana la última”. En este módulo se evita la ambigüedad mediante condiciones mutuamente excluyentes.

```
<style name="UnidadesCondicional" style="Dato" isBold="true">
  <conditionalStyle>
    <conditionExpression><![CDATA[
      {$F{unidades_vendidas} != null
      && $P{umbralUnidades} != null
      && {$F{unidades_vendidas}.intValue() >= $P{umbralUnidades}.intValue()}
    ]]></conditionExpression>
    <style forecolor="#1B5E20"/>
  </conditionalStyle>
  <conditionalStyle>
    <conditionExpression><![CDATA[
      {$F{unidades_vendidas} != null
      && $P{umbralUnidades} != null
      && {$F{unidades_vendidas}.intValue() >= 3
      && {$F{unidades_vendidas}.intValue() < $P{umbralUnidades}.intValue()}
    ]]></conditionExpression>
    <style forecolor="#1D5D88"/>
  </conditionalStyle>
  <conditionalStyle>
    <conditionExpression><![CDATA[
      {$F{unidades_vendidas} == null
```

XML

```

    || ${unidades_vendidas}.intValue() < 3
  ]]></conditionExpression>
  <style forecolor="#9D3429" />
</conditionalStyle>
</style>

```

El atributo `style="Dato"` referencia el estilo base del que hereda este estilo. Las tres reglas representan tramos distintos: alto, medio y bajo/sin ventas; así una fila solo entra en un tramo de color.

Bloque 4 — Lógica condicional sobre campos, parámetros y variables

La lógica condicional puede combinar campos, parámetros y variables en una misma expresión. Una condición puede comprobar el valor de un campo del registro actual, el valor de un parámetro proporcionado por el usuario y el valor acumulado de una variable. La combinación de los tres tipos de referencias permite construir condiciones que dependen simultáneamente del dato, de la instrucción y del estado del informe.

```

<printWhenExpression><![CDATA[
  ${mostrarDetalle}.booleanValue()
  && ${unidades_vendidas} > ${umbralUnidades}
  && ${TotalImporte} > 0
]]></printWhenExpression>

```

Línea 2	<code>\${mostrarDetalle}.booleanValue()</code> → primera condición. Comprueba el valor de un parámetro.
Línea 3	<code>&& \${unidades_vendidas} > \${umbralUnidades}</code> → segunda condición. Comprueba el valor de un campo contra un parámetro.
Línea 4	<code>&& \${TotalImporte} > 0</code> → tercera condición. Comprueba el valor de una variable.

La combinación de condiciones requiere respetar el orden estructural del JRXML. En este informe se declaran primero estilos y parámetros; después `queryString`; a continuación los fields que describen las columnas devueltas, luego las variables y finalmente las bandas. Las expresiones de variables pueden referenciar fields y parámetros ya definidos; las bandas consumen parámetros, fields y variables.

ORDEN DE DECLARACIÓN EN EL JRXML

1. Properties
2. Styles
3. Parameters
4. QueryString
5. Fields
6. Variables
7. Title y demás bandas

El orden respeta el esquema JRXML y sitúa cada referencia en un contexto donde el motor ya conoce sus dependencias.

Qué representa el diagrama: el orden de declaración de las secciones del JRXML. Cada sección solo puede referenciar las anteriores.

Por qué es relevante: permite organizar el archivo JRXML de forma coherente y evitar errores de resolución.

Bloque 5 — Visibilidad condicional de columnas completas

La visibilidad condicional de una columna completa requiere aplicar `printWhenExpression` al encabezado de la columna en la banda `columnHeader` y a cada uno de los campos de la columna en la banda `detail`. La condición debe ser la misma en ambos casos para que la columna aparezca o desaparezca de forma coherente. Si se aplica solo al encabezado, los datos de la columna siguen apareciendo sin su rótulo. Si se aplica solo a los datos, los datos desaparecen pero el encabezado permanece. La coherencia entre las dos bandas es condición necesaria para que el efecto sea correcto.

```

<columnHeader>
  <band height="25">
    <staticText>
      <reportElement x="440" y="5" width="115" height="15" uuid="..." />
      <printWhenExpression><![CDATA[${mostrarDetalle}.booleanValue()]]></printWhenExpression>
      <textElement textAlignment="Center" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10" isBold="true"/>
      </textElement>
      <text><![CDATA[Clasificación]]></text>
    </staticText>
  </band>
</columnHeader>
<detail>
  <band height="90">
    <textField>

```

```

<reportElement x="440" y="50" width="115" height="15" uuid="..."/>
<printWhenExpression><![CDATA[{P{mostrarDetalle}.booleanValue()}]]></printWhenExpression>
<textElement textAlignment="Center" verticalAlignment="Middle">
  <font fontName="DejaVu Sans" size="9"/>
</textElement>
<textFieldExpression><![CDATA[{F{precio_medio} > 22 ? "Premium" : ...}]]></textFieldExpression>
</textField>
</band>
</detail>

```

Línea 2-6	staticText con el encabezado Clasificación. La propiedad printWhenExpression controla su visibilidad según el parámetro mostrarDetalle.
Línea 12-16	textField con la clasificación en la banda detail. La misma condición controla su visibilidad.

La visibilidad condicional de columnas completas resulta útil cuando el usuario puede solicitar informes con distintos niveles de detalle. Un informe con `mostrarDetalle` verdadero muestra todas las columnas. Un informe con `mostrarDetalle` falso muestra solo las columnas principales. La misma plantilla produce dos informes distintos sin necesidad de duplicar el diseño. Esta técnica es una de las más utilizadas en los informes empresariales porque permite adaptar la presentación a las preferencias del usuario sin multiplicar el número de plantillas.

INFORME CON `mostrarDetalle=true`

Título	Unidades	Importe	Categoría	Clasificación	...
Libro	8	159,60 €	Novela	Estándar	...

INFORME CON `mostrarDetalle=false`

Título	Unidades	Importe
Libro	8	159,60 €

Qué representa el diagrama: el mismo informe con `mostrarDetalle` verdadero y falso. Las columnas adicionales aparecen solo cuando el parámetro lo indica.

Por qué es relevante: permite construir informes adaptables con una única plantilla y un parámetro de control.

Resumen rápido de la teoría

- Los operadores lógicos `&&`, `||` y `!` permiten combinar condiciones simples.
- La precedencia de los operadores puede modificarse con paréntesis.
- `printWhenExpression` controla la visibilidad de bandas y elementos individuales.
- La visibilidad condicional es en cascada: la banda controla todos sus elementos.
- Los estilos condicionales se declaran con `conditionalStyle` y se evalúan en orden.
- La lógica condicional combina campos, parámetros y variables.
- La visibilidad de columnas completas requiere aplicar la misma condición al encabezado y a los datos.
- El orden de declaración en el JRXML determina la disponibilidad de las referencias.

Punto 4.6 — Parámetros en consultas SQL

Módulo, proyecto y objetivos de aprendizaje

Módulo: 4 — Parámetros y lógica (3 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial

Punto: 4.6 — Parámetros en consultas SQL

Objetivos de aprendizaje

- Comprender el mecanismo de sustitución de parámetros en las consultas SQL.
- Diferenciar `$P{}` para valores enlazados, `$X{}` para cláusulas parametrizadas y `$P!{}` para sustitución textual directa.
- Construir filtros parametrizados con `LIKE`, `IN` y rangos de fechas.
- Prevenir la inyección SQL mediante el uso correcto de los parámetros.
- Combinar varios parámetros en una consulta con lógica condicional en SQL.
- Documentar las consultas parametrizadas del proyecto EditorialReports.

Parte teórica

Bloque 1 — Cómo se enlazan los parámetros en una consulta SQL

En JasperReports, un parámetro de consulta escrito como `$P{nombre}` **no se pega como texto dentro del SQL**. El query executer transforma esa referencia en un marcador `?` de una sentencia JDBC preparada y entrega el valor al `PreparedStatement` por separado. Por tanto, el tipo Java declarado en el parámetro sigue siendo importante, pero no porque JasperReports tenga que añadir manualmente comillas al texto SQL, sino porque JDBC debe enlazar el valor con el tipo adecuado.

```
<parameter name="categoria" class="java.lang.String"/>
<queryString language="sql">
  <![CDATA[
    SELECT titulo, precio
    FROM libros
    WHERE categoria = $P{categoria}
  ]]>
</queryString>
```

XML

Conceptualmente, el motor prepara una sentencia equivalente a:

```
SELECT titulo, precio
FROM libros
WHERE categoria = ?

bind #1 -> "Novela"
```

TEXT

La consulta y el valor viajan separados. Si `categoria` vale `null`, JDBC enlaza un valor SQL nulo; por eso los filtros opcionales de este curso usan una condición como `($P{categoria} IS NULL OR l.categoria = $P{categoria})`. No debe imaginarse esa expresión como una sustitución literal del texto `$P{categoria}` por la palabra `NULL`.

Qué aporta al proyecto: permite parametrizar valores sin construir SQL concatenando cadenas y mantiene el contrato de tipos entre Java, JasperReports y JDBC.

Bloque 2 — Diferencia entre `$P{}`, `$X{}` y `$P!{}`

JasperReports ofrece tres mecanismos distintos y conviene no mezclarlos:

- **`$P{nombre}`:** representa un **valor**. En consultas JDBC termina como un marcador `?` y se enlaza mediante `PreparedStatement`.
- **`$X{función, columna, parámetro}`:** ejecuta una **función de cláusula** de JasperReports. Se usa cuando la forma de una condición depende del valor, por ejemplo para crear un `IN` con una colección. Los valores generados siguen enlazándose como parámetros JDBC.
- **`$P!{nombre}`:** realiza **sustitución textual directa** antes de preparar la consulta. Sirve únicamente para fragmentos estructurales que la aplicación controle estrictamente; no debe recibir texto arbitrario del usuario.

TEXT

```
$P{categoria}
SQL preparado: WHERE categoria = ?
bind: "Novela"

${IN, categoria, categoriasLista}
SQL generado: WHERE categoria IN (?, ?, ...)
binds: cada elemento de la colección

$P!{ordenControlado}
Inserta texto en la consulta antes de prepararla.
```

Esta distinción es fundamental para entender seguridad y depuración. Decir que `${X}` es “sustitución directa” es incorrecto: la sustitución textual directa es `$P!{}`.

Bloque 3 — Filtro parametrizado con LIKE

El operador `LIKE` puede combinarse con un valor enlazado. En `SQLite`, `EditorialReports` construye los comodines en la propia expresión SQL y mantiene el texto del usuario como bind parameter:

XML

```
<parameter name="textoBusqueda" class="java.lang.String"/>
<queryString language="sql">
  <![CDATA[
    SELECT titulo
    FROM libros
    WHERE ($P{textoBusqueda} IS NULL
      OR $P{textoBusqueda} = ''
      OR titulo LIKE '%' || $P{textoBusqueda} || '%')
  ]]>
</queryString>
```

Para el valor `sol`, la estructura sigue siendo estable: el texto `sol` no pasa a formar parte de la sintaxis SQL. `SQLite` concatena los comodines con el valor enlazado y busca títulos que contengan esa secuencia.

Otra estrategia válida consiste en construir `"%"+texto+"%"` en Java y usar simplemente `titulo LIKE $P{patronTitulo}`. Ambas mantienen el valor separado de la estructura SQL; la elección depende de dónde se quiera concentrar la lógica de construcción del patrón.

Bloque 4 — \${X{IN, ...}} con colecciones

Una colección no debe tratarse como un único parámetro escalar dentro de `IN`. Para ello `JasperReports` proporciona la función de cláusula `IN`:

XML

```
<parameter name="categoriasLista" class="java.util.Collection"/>
<queryString language="sql">
  <![CDATA[
    SELECT titulo, categoria
    FROM libros
    WHERE ${X{IN, categoria, categoriasLista}}
  ]]>
</queryString>
```

Si la colección contiene, por ejemplo, `Novela` y `Poesía`, `JasperReports` construye una condición equivalente a `categoria IN (?, ?)` y enlaza los dos valores. La función también contempla valores nulos dentro de la colección.

Una colección nula o vacía **no se convierte en una lista SQL inválida**. En ese caso `JasperReports` genera una cláusula de resultado constante. El resultado puede controlarse mediante el cuarto argumento opcional de la función y mediante la propiedad `net.sf.jasperreports.sql.clause.in.novalues.result`. Por eso una práctica correcta debe explicar la semántica de ausencia de valores en lugar de mostrar SQL inválido.

En el checkpoint 4.6 el escenario base pasa explícitamente las cuatro categorías existentes. Así se conservan los 14 títulos mientras se demuestra el mecanismo `${X{IN, ...}}`.

Bloque 5 — Prevención de inyección SQL

La regla principal es mantener separados **estructura SQL** y **valores**. Con `$P{}`, `JasperReports/JDBC` usa una sentencia preparada. No es necesario ni correcto explicar el mecanismo como un escape manual de comillas.

Entrada de prueba:

TEXT

```
sol' OR '1'='1
```

Con el filtro del checkpoint:

SQL

```
titulo LIKE '%' || ${textoBusqueda} || '%'
```

la estructura preparada sigue siendo equivalente a:

TEXT

```
titulo LIKE '%' || ? || '%'  
bind -> sol1 OR '1'='1'
```

El contenido malicioso se trata como **dato**, no como parte de la consulta. `$X{}` debe limitarse a las funciones de cláusula previstas por JasperReports y `$P!{}` solo debe usarse con fragmentos estructurales seleccionados por la propia aplicación desde opciones cerradas.

Buenas prácticas de EditorialReports:

1. usar `$P{}` para valores escalares;
2. usar `$X{}` para cláusulas dinámicas soportadas, como IN;
3. evitar `$P!{}` con cualquier texto no confiable;
4. no concatenar manualmente valores del usuario dentro del SQL;
5. probar explícitamente nulos, colecciones vacías y cadenas con caracteres especiales.

Resumen rápido de la teoría

- JasperReports prepara la consulta y enlaza los valores de `$P{}` mediante JDBC; las funciones `$X{}` generan cláusulas parametrizadas cuando la estructura depende de una colección o condición.
- La sintaxis `$P{}` usa parámetros enlazados (`PreparedStatement`) y mantiene separados el SQL y los valores.
- La sintaxis `$X{}` es una función de cláusula para construir de forma controlada fragmentos como IN, enlazando los valores; la sustitución textual directa corresponde a `$P!{}` y debe restringirse a fragmentos SQL controlados.
- El operador LIKE permite búsquedas parciales con comodines.
- El operador IN permite búsquedas por listas de valores.
- La sintaxis `$X{IN, columna, parámetro}` genera dinámicamente la lista.
- La prevención de inyección SQL se basa en el uso correcto de `$P{}` y en la validación de valores.
- La documentación de las consultas parametrizadas es una buena práctica.